

Assignment date: 16th may 2026

Machine learning

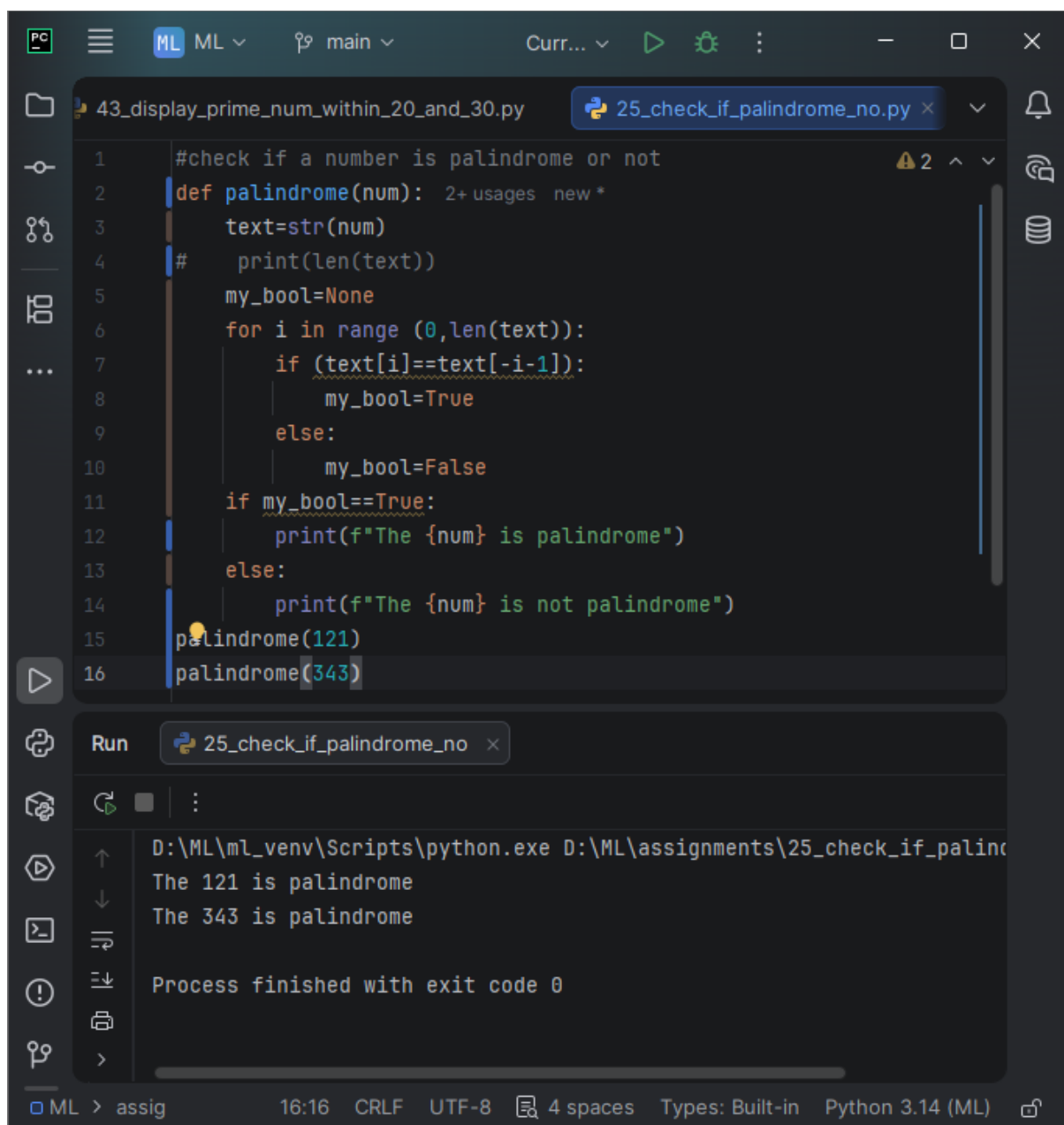
Python

ABHAY RAJ

1Q. Check if a no. is palindrome:

a. 121

b. 343



The screenshot shows a Python IDE with a file named `25_check_if_palindrome_no.py` open. The code defines a function `palindrome(num)` that checks if a number is a palindrome. It converts the number to a string, iterates through the characters from both ends, and compares them. If all characters match, it prints "The {num} is palindrome"; otherwise, it prints "The {num} is not palindrome". The function is called with `121` and `343`.

```
1 #check if a number is palindrome or not
2 def palindrome(num):
3     text=str(num)
4     # print(len(text))
5     my_bool=None
6     for i in range (0,len(text)):
7         if (text[i]==text[-i-1]):
8             my_bool=True
9         else:
10            my_bool=False
11    if my_bool==True:
12        print(f"The {num} is palindrome")
13    else:
14        print(f"The {num} is not palindrome")
15    palindrome(121)
16    palindrome(343)
```

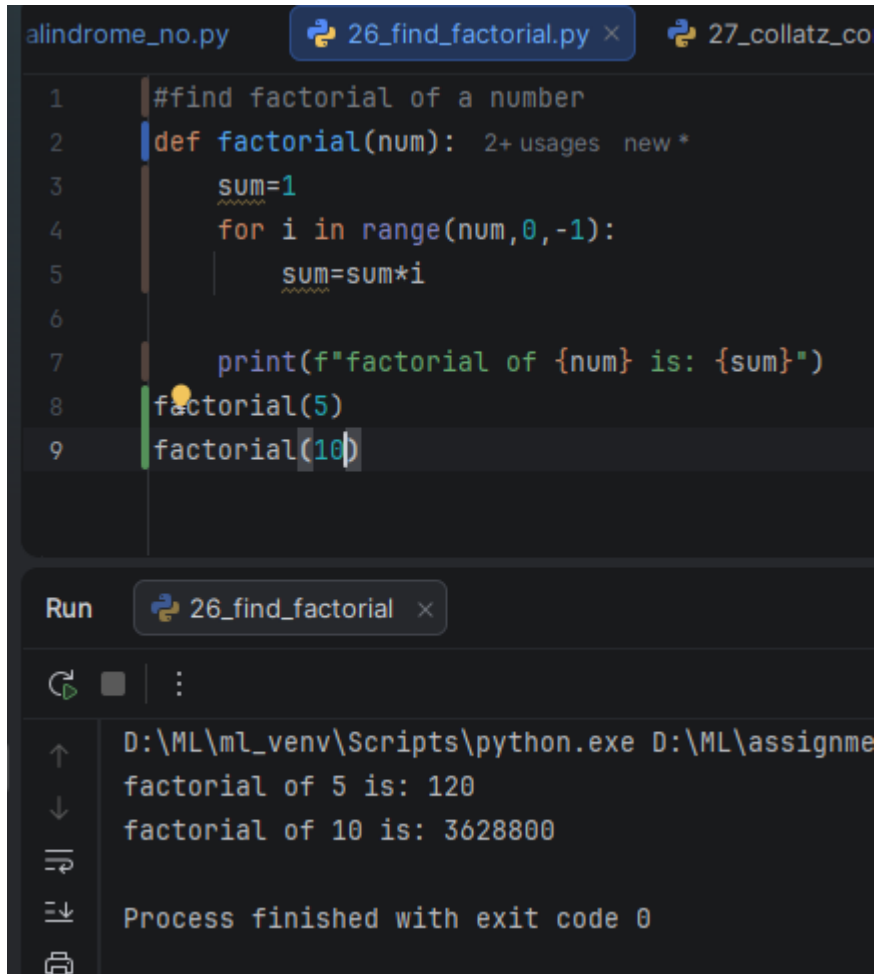
The Run console shows the output of the program:

```
D:\ML\ml_venv\Scripts\python.exe D:\ML\assignments\25_check_if_palindrome_no.py
The 121 is palindrome
The 343 is palindrome
Process finished with exit code 0
```

The status bar at the bottom indicates the file encoding is UTF-8, the line ending is CRLF, and the Python version is 3.14 (ML).

Q2. Check factorial of a number

- a. 5
- b. 10



The screenshot shows a Python IDE with three tabs: 'alindrome_no.py', '26_find_factorial.py' (active), and '27_collatz_co'. The active tab contains the following code:

```
1 #find factorial of a number
2 def factorial(num):
3     sum=1
4     for i in range(num,0,-1):
5         sum=sum*i
6
7     print(f"factorial of {num} is: {sum}")
8 factorial(5)
9 factorial(10)
```

Below the code editor, there is a 'Run' button and a terminal window. The terminal shows the output of the executed code:

```
D:\ML\ml_venv\Scripts\python.exe D:\ML\assignme
factorial of 5 is: 120
factorial of 10 is: 3628800
Process finished with exit code 0
```

Q3. Collatz Conjecture: Generate a sequence until it reaches 1.

- a. n=6

```
alindrome_no.py  26_find_factorial.py  27_collatz_conjecture.py
1      n=6
2      while True:
3          if n==1:
4              break
5          elif n%2==0:
6              n/=2
7              print(int(n), end=", ")
8          elif n%2!=0:
9              n=(n*3)+1
10             print(int(n), end=", ")
11
```

Run 27_collatz_conjecture x

D:\ML\ml_venv\Scripts\python.exe D:\ML\assignments\3, 10, 5, 16, 8, 4, 2, 1,
Process finished with exit code 0

Q4. Armstrong no. check.

Num= 153

```

n=153
num=153
number=153
count=0

#getting no. of digits
while True:

    count+=1
    n=n//10
    if n<=0:
        break
print("Numbers of digit: ",count)

sum=0
while True:
    p=num%10
    num//=10
    c=p**count
    sum+=c
    if num<=0:
        break

if sum==number:
    print("The given number is armstrong")

```

output

```

D:\ML\ml_venv\Scripts\python.exe D:\
Numbers of digit:  3
The given number is armstrong

Process finished with exit code 0

```

Q5. Print right-angled triangle no. pattern using a loop.

Given height=5

```
n=5
for i in range(0,n+1):
    for j in range(0,i):
        print(i,end="")
    print()

29_num_pattern_right_angled_tri x
```

D:\ML\ml_venv\Scripts\python.exe D:\ML\

1
22
333
4444
55555

Process finished with exit code 0

Q6. Using above question print decreasing pattern.

```
n=5
for i in range(n,0,-1):
    for j in range(i,0,-1):
        print(i,end="")
    print()

n 30_decreasing_pattern_right_ang_tri x
```

D:\ML\ml_venv\Scripts\python.exe D:\M

55555
4444
333
22
1

Process finished with exit code 0

Q7. Print alternate no. pattern

(Range 1 to 20)

```
30_decreasing_pattern_right_ang_tri.py 31_alt_num_pattern_tri.py x
n=20
for i in range(0,n+1,2):
    for j in range(0,i):
        print(i,end=" ")
    print()
```

D:\ML\ml_venv\Scripts\python.exe D:\ML\assignments\31_alt_num_

```
2 2
4 4 4 4
6 6 6 6 6 6
8 8 8 8 8 8 8 8
10 10 10 10 10 10 10 10 10 10
12 12 12 12 12 12 12 12 12 12 12 12
14 14 14 14 14 14 14 14 14 14 14 14 14 14
16 16 16 16 16 16 16 16 16 16 16 16 16 16 16 16
18 18 18 18 18 18 18 18 18 18 18 18 18 18 18 18 18 18
20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20 20

Process finished with exit code 0
```

Q8. Print Alphabet pyramid (A,BB,CCC)

```
i.py 31_alt_num_pattern_tri.py 32_alphabet_tri_pattern.py x
n=3
for i in range(n):
    #65 is ASCII value for 'A'
    char=chr(65+i)
    row=" ".join([char]*(i+1))

    #calculate the exact total width of the bottom row
    max_width = 2 * n - 1

    #center the row perfectly
    print(row.center(max_width))

n 32_alphabet_tri_pattern x
:
D:\ML\ml_venv\Scripts\python.exe D:\ML\assignments\32_alph
A
B B
C C C

Process finished with exit code 0
```

Q9. print hollow square pattern

Size=5

```
1 size=5
2
3 for i in range(size):
4     if i==0 or i==size-1:
5         print("* " * size)
6     else:
7         print("* " + " " * (size+2) + "* ")
```

Run 33_hollow_sq_pattern x

D:\ML\ml_venv\Scripts\python.exe D:\ML\ass

```
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

Process finished with exit code 0

Q10. print pyramid pattern of stars

Rows=5

```
ern.py 33_hollow_sq_pattern.py 34_pyramid_pattern_star.py
1 rows=5
2 for i in range(rows):
3     row=("* "*(i+1))
4
5     max_width=(3*rows)-1
6     print(row.center(max_width))

Run 34_pyramid_pattern_star x
D:\ML\ml_venv\Scripts\python.exe D:\ML\assignments\34_
    *
   * *
  * * *
 * * * *
* * * * *

Process finished with exit code 0
```

Q11. Print full multiplication table (1 to 100)

input	
<pre>n=100 for i in range(1,100+1): for j in range(1,10+1): print(f"{i} * {j} = {j*i}") print("\n")</pre>	
output	

```
98 * 4 = 392
98 * 5 = 490
98 * 6 = 588
98 * 7 = 686
98 * 8 = 784
98 * 9 = 882
98 * 10 = 980

99 * 1 = 99
99 * 2 = 198
99 * 3 = 297
99 * 4 = 396
99 * 5 = 495
99 * 6 = 594
99 * 7 = 693
99 * 8 = 792
99 * 9 = 891
99 * 10 = 990

100 * 1 = 100
100 * 2 = 200
100 * 3 = 300
100 * 4 = 400
100 * 5 = 500
100 * 6 = 600
100 * 7 = 700
100 * 8 = 800
100 * 9 = 900
100 * 10 = 1000

Process finished with exit code 0
```

Q12. Dictionary Filter:

Extract pairs where value exceeds a threshold.

Given input:

Scores={"Abhay": 85, "Sachin":70}

threshold=75

```
scores={"Abhay":85,"Sachin":70}
threshold = 75

filtered_score = {key: value for key, value in scores.items() if value > threshold}

print("Filtered Dictionary: ", filtered_score)
```

36_dict_filter_threshold

D:\ML\ml_venv\Scripts\python.exe D:\ML\assignments\36_dict_filter
Filtered Dictionary: {'Abhay': 85}

Process finished with exit code 0

Q13. Remove duplicates without set

Input: [1,2,2,3,4,4,4,5]

```
input_list=[1,2,2,3,4,4,4,5]
new_list=[]

for item in input_list:
    if item not in new_list:
        new_list.append(item)
print(new_list)
```

37_removing_duplicates_from_list_without_sets

D:\ML\ml_venv\Scripts\python.exe D:\ML\assignments\
[1, 2, 3, 4, 5]

Process finished with exit code 0

Q14. word frequency counter

Input: text="apple banana apple orange banana apple"

```
text= "apple banana apple orange banana apple"

list1=text.split()

temp=[]
count=0
print(list1)
for i in range (len(list1)):
    if list1[i] not in temp:
        temp.append(list1[i])
    else :
        count+=1

print(temp)
print(count)
```

39_word_frequency_count

```
D:\ML\ml_venv\Scripts\python.exe D:\ML\assignments\39_word_fr
['apple', 'banana', 'apple', 'orange', 'banana', 'apple']
['apple', 'banana', 'orange']
3

Process finished with exit code 0
```

Q15. Display Fibonacci series upto 10 terms.

Input: n_terms=10

```
39_word_frequency_count.py 40_fibonacci_series.py x 41_check

n_terms=10
#0,1,1,2,3,5,8,13,21

my_list=[None]*10
print(my_list)
print(len(my_list))

my_list[0], my_list[1]=0,1
for i in range(2,len(my_list)):
    if i==1:
        my_list[i]=1
        continue
    my_list[i]=my_list[i-2]+my_list[i-1]

print(my_list)

un 40_fibonacci_series x

D:\ML\ml_venv\Scripts\python.exe D:\ML\assignments\40_fibonacci_series.py
[None, None, None, None, None, None, None, None, None, None]
10
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

Process finished with exit code 0
```

Q16. check whether 28 is a perfect number or not.

```
n=28
sum=0
for i in range(1,n):
    if n%i==0:
        sum=sum+i
if sum==n:
    print(f"{n} is perfect number")
```

41_check_perfect_num_28 x

D:\ML\ml_venv\Scripts\python.exe D:\ML\assignments\41
28 is perfect number

Process finished with exit code 0

Q17. binary to decimal conversion using loop.

Input : binary_str="1101"

```
binary_str="1101"
num=0
for bit in (binary_str):
    num = num * 2 + int(bit)
print(num)
```

42_binary_int x

D:\ML\ml_venv\Scripts\python.exe D:\ML\ass
13

Process finished with exit code 0

Q18. display all prime no. within 20 and 30

```
def is_prime(n: {__le__, __mod__}): 1 usage ABHA
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

primes = []
for n in range(20, 30):
    if is_prime(n):
        primes.append(n)
print(primes)
```

43_display_prime_num_within_20_and_30 x

D:\ML\ml_venv\Scripts\python.exe D:\ML\assignment
[23, 29]

Process finished with exit code 0